

An experimental comparison of ER and UML class diagrams for data modelling

Andrea De Lucia · Carmine Gravino · Rocco Oliveto ·
Genoveffa Tortora

Published online: 11 December 2009
© Springer Science+Business Media, LLC 2009

Editor: Erik Arisholm

Abstract We present the results of three sets of controlled experiments aimed at analysing whether UML class diagrams are more comprehensible than ER diagrams during data models maintenance. In particular, we considered the support given by the two notations in the comprehension and interpretation of data models, comprehension of the change to perform to meet a change request, and detection of defects contained in a data model. The experiments involved university students with different levels of ability and experience. The results demonstrate that using UML class diagrams subjects achieved better comprehension levels. With regard to the support given by the two notations during maintenance activities the results demonstrate that the two notations give the same support, while in general UML class diagrams provide a better support with respect to ER diagrams during verification activities.

Keywords Controlled experiments · Entity-relation diagrams · UML class diagrams · Design notations · Comprehension · Maintenance · Verification

The work described in this paper is supported by the project METAMORPHOS (MEthods and Tools for migrAting software systeMs towards web and service Oriented aRchitectures: exPerimental evaluation, usability, and tecHnOlogy tranSfer), funded by MiUR (Ministero dell'Università e della Ricerca) under grant PRIN-2006-2006098097.

A. De Lucia · C. Gravino · R. Oliveto (✉) · G. Tortora
Department of Mathematics and Informatics, University of Salerno,
via Ponte don Melillo, 84084 Fisciano, Salerno, Italy
e-mail: roliveto@unisa.it

A. De Lucia
e-mail: adelucia@unisa.it

C. Gravino
e-mail: gravino@unisa.it

G. Tortora
e-mail: tortora@unisa.it

1 Introduction

A data model is a set of concepts that can be used to describe both the structure of and the operations on a database (Navathe 1992). It represents the output of data modelling (or conceptual design), an activity that aims at creating a conceptual schema in a diagrammatic form and facilitating the communication between developers and users (Navathe 1992). Once approved by the users, the conceptual schema is converted into a specific database schema, depending on the data model and the database management system (DBMS) used for the implementation. This conversion is quite simple, as it is an algorithmic and automatic process. The major problem is to create a good conceptual schema that is semantically correct and comprehensible (Shoval and Shiran 1997).

Many graphical notations have been proposed in the literature to represent data models. Since 1976, Entity-Relationship (ER) and its extensions are the most used notations for database conceptual modelling and still remains the *de facto* standard (Navathe 1992). The success of the Object-Oriented (OO) approach for programming has encouraged the use of this approach also for database modelling (Shoval and Frummermann 1994). In particular, UML class diagrams can be used to represent the conceptual schema of a software system (Rumbaugh et al. 2004). In addition to static (structural) constructs of the model whose representation of data structure is somewhat equivalent to Extended ER (EER) representation (e.g., object classes considered equivalent to entity and relationship types) the OO approach models system behaviour through “methods” that are attached to the object classes. If there are no doubts about the advantages of the OO approach for programming (see for instance software reuse and information hiding), the superiority of the OO approach in earlier stages of software development, i.e., system analysis and data modelling, has not been completely proven yet. Very often the choice of which notation has to be used comes from industrial standards and practices independently of the effectiveness of the employed notations. Moreover, while UML is becoming a *de facto* standard for the analysis and design of software systems, it is not exploited with the same success for modelling databases. Indeed, nowadays ER remains the most used notation to model databases and in some cases it complements UML in the design of software systems. In a recent small industrial survey carried out with our industrial we found that 4 of 13 contacted software companies usually employ both ER and UML class diagrams to represent the same database (De Lucia et al. 2008b). Clearly, this can be a problem during the evolution of the data models since more effort is required to maintain the models and their implementation up-to-date.

In the last decades several controlled experiments have been carried out to compare the ER and OO notations from a designer perspective (Shoval and Shiran 1997; Shoval and Frummermann 1994; Bock and Ryan 1993; Palvia et al. 1992). Such empirical evaluations aimed at discovering strengths and weaknesses of design notations, determining the costs of applying or the benefits of using a particular notation. The results do not demonstrate the actual benefits given by the ER diagrams to justify the real need of preferring this notation for data modelling.

Recently, we compared ER and UML class diagrams from a maintainer perspective (De Lucia et al. 2008b, c). In particular, in De Lucia et al. (2008c) we reported the results of a controlled experiment and a replication aimed at analysing whether UML class diagrams are more comprehensible than ER diagrams. The two experiments

involved undergraduate and graduate students, respectively. The results showed that using UML class diagrams subjects achieved significantly better comprehension levels in both the experiments. In De Lucia et al. (2008b) the two notations were compared during activities aimed at comprehending the change to perform to meet a change request. The results demonstrated that in this case the two notations provide the same support.

In this paper we complete the empirical comparison of ER and UML class diagrams evaluating (i) the comprehensibility of the two notations from a customer perspective and (ii) analysing whether UML class diagrams are more effective than ER diagrams in the comprehension and detection of defects contained in data models. We also report and discuss the results of our previous controlled experiments (De Lucia et al. 2008b, c) to have a wide comparison of ER and UML class diagrams during comprehension, maintenance, and verification activities. We focus the attention only on these three activities, since they correspond to typical expectations of people using these two notations to represent conceptual data models. In particular, the higher abstraction level of models should help to understand a system, modify it, and avoid defects in the early stages of development (Briand et al. 2005). Thus, the specific contributions of this paper with respect to (De Lucia et al. 2008b, c) are represented by a complete empirical comparison of the support given by UML class diagrams during comprehension, maintenance, and verification activities from the perspective of business analysts and customers with respect to ER diagrams. Indeed, our conjecture is that UML class diagrams are easier to understand and maintain than ER diagrams due to their more concise graphical notation. Clearly, if such a conjecture is supported by empirical results, then the suggestion is to evaluate the possibility to move from ER diagrams to UML diagrams beginning from academic courses. Probably, this would also facilitate the migration from ER diagrams to UML class diagrams in industrial contexts.

The rest of the paper is organised as follows. Section 2 provides details of the design of the experiment, including the experiment definition, context selection, hypotheses formulation, and identification of experimental factors. Section 3 reports and discusses the achieved results. Section 4 discusses the threats to validity that could affect the results achieved in our controlled experiments, while Section 5 summarises the lessons learned. Related work is compared with our study in Section 6 and concluding remarks and directions for future work are given in Section 7.

2 Experimental Method

This section describes in detail the definition, design, and settings of all the controlled experiments we performed.¹ We replicated each experiment at least once, and the design, the material and the procedure of the replications were exactly the same as the main experiments. Subjects represented the only substantial difference among the experiment and the replications.

¹See De Lucia et al. (2008a) for the complete material used in the three sets of controlled experiments.

The experimental method followed the guidelines by Wohlin et al. (2000) and Juristo and Moreno (2001). Moreover, according to the two-dimensional classification scheme by Basili et al. (1986), we performed blocked subject-project studies, as we examined objects across a set of subjects and a set of projects.

2.1 Experiment Definition

The *goal* of our experimentation was to analyse whether UML class diagrams are more comprehensible than ER diagrams during different kinds of activities:

- *Understanding and interpreting the data models*: people with different roles in the project have to understand data models at different stages of software development. In particular, during the analysis phase the data models can be useful to support the requirement elicitation phase. The requirement elicitation involves software engineers as well as customers that probably do not have any background on design notations. Thus, a comprehensive notation is really desirable to avoid misunderstanding in the requirement analysis phase. Indeed, avoiding misunderstandings is very important, because they can lead to the introduction of errors that might be very expensive to remove in the later phases of the software development.
- *Comprehending the change to perform on data models*: a software system undergoes continual changes during its life-cycle aiming at improving performances or other attributes, or adapting the product to a modified environment. Each type of change may require a modification to the data model as well. Thus, a notation that helps to point out the impact of change easily, correctly, and quickly is desirable.
- *Comprehending/detecting defects in the data models*: defects are likely to exist in analysis and design models, including data models. They can be due for instance to a misunderstanding of the requirements or miscommunication among team members (Briand et al. 2005). For this reason, models undergo a review process aiming at detecting defects. A design notation should support this activity helping to easily verify the coherence of the model with the system requirements.

In the rest of the paper we will refer to these activities as comprehension, maintenance, and verification activities, respectively. To assess the support given by UML class diagrams and ER diagrams during these activities we carried out three sets of controlled experiments:

1. *Comprehension*: a controlled experiment and two replications were carried out to assess whether UML class diagrams provide a better support than ER diagrams in the comprehension of data models.
2. *Maintenance*: a controlled experiment and a replication were carried out to assess whether UML class diagrams provide a better support than ER diagrams in the comprehension of the change to perform on the data model to meet a change request.
3. *Verification*: a controlled experiment and a replication were carried out to assess whether UML class diagrams provide a better support than ER diagrams in the detection of defects in a data model.

The *perspective* of our experimentation was both of *business analysts*, evaluating the possibility of adopting a design notation within their own organisation, depending on the skills of the involved human resources; and (ii) of *customers*, evaluating how effective UML class diagrams are during comprehension activities. The subjects of our study were selected according to this perspective.

2.2 Experimental Context

In the following subsections we describe the context of the controlled experiments, focusing the attention on subjects and objects.

2.2.1 Subjects

The controlled experiments were executed at the University of Salerno (Italy) and involved students having different academic backgrounds and, consequently, different levels of experience on ER and UML diagrams:

- *zero-knowledge students*, i.e., fresher B.Sc. students that were starting their academic career when the experiment was performed;
- *bachelor students*, i.e., 2nd year B.Sc. students that attended Programming and Databases courses in the past and were attending the Software Engineering course when the experimentation was performed. The design notation used in the Software Engineering course is UML;
- *master students*, i.e., 1st year M.Sc. students that attended advanced courses of Programming and Software Engineering in the past and were attending an advanced Databases course when the experimentation was performed. The design notation used in the Databases course is ER.

With regard to the ethics of the experiment, it is important to note that the experiments were part of a series of extra-laboratory exercises conducted within the Software Engineering and Databases courses and students were not evaluated on their performances. Moreover, these laboratory exercises were not part of the courses and students were free to participate or not.

Table 1 reports the number of subjects involved in each experiment. Within each experiment, all students were from the same class with a comparable level of background but different levels of ability. Excepting the fresher students, that had no experience in software design and development, all the students had knowledge of both software development and software documentation, including database design and documentation. Moreover, 2nd year B.Sc. and 1st year M.Sc. students had a fairly good knowledge of both ER and UML diagrams, even if master students had more experience than bachelor students on the design methods. For this reason, zero-knowledge students are supposed to represent customers, while bachelor and master

Table 1 Subjects involved in the experimentation

	Main experiment	1st replication	2nd replication
Comprehension	40 undergraduate students	30 master students	68 fresher students
Maintenance	40 undergraduate students	30 master students	–
Verification	40 undergraduate students	30 master students	–

Table 2 Characteristics of the data models used in each set of experiments

Experiment	System	# entities	# attributes	# relationships
Comprehension	ADAMS-TeamManagement	6	21	5
	EasyClinic-BookingManagement	6	18	5
Maintenance	Company	7	17	5
	University	7	20	6
Verification	ADAMS-EventManagement	7	20	5
	EasyClinic-VisitManagement	9	22	7

students are supposed to represent junior business analysts with two different levels of experience.

Note that since maintenance and verification of data models are difficult activities and require deep knowledge of the design method, fresher students were only involved in the controlled experiment concerned with the comprehension and interpretation of data models.

2.2.2 Objects

In the experimentation we used the data models of the following systems:

- ADAMS, a web-based artefact management system developed at the University of Salerno (De Lucia et al. 2004).
- Company, a software system implementing all the operation required to manage the projects conducted by a company;
- EasyClinic, a software system implementing all the operations required to manage a medical doctor's office;
- University, a software system implementing all the operations required to manage university courses.

In particular, for each set of experiments we exploited two different data models represented by using ER and UML class diagrams. Table 2 shows the characteristics of the data models we employed in the experiments. The selection of the objects for each controlled experiment was performed ensuring that the data models had a comparable level of complexity. For this reason, we extracted sub-diagrams of comparable size from the original data models according to the “*the rule of seven*” given by Miller (1956) to build comprehensible graphical diagrams.² In the context of our experimentation we applied such a rule to select data models easy to comprehend. This was necessary because (i) the experiment was designed to be performed in a limited amount of time and (ii) a simple data model is preferred to a more complex data model since the latter might influence the comprehension activities. Indeed, a simple diagram allowed us to focus the attention only on the notation used to represent the diagram. For this reason, we identified the four medium/large software systems described above with a data model composed of a high number of entities (to have systems comparable with industrial projects) and we extracted—according to the Miller’s rule—a subdiagram with only 7 ± 2 entities of the original data models.

²The rule of seven is the generally accepted claim that people can hold approximately seven chunks or units of information in their short-term memory at a time (Miller 1956).

In particular, in the set of experiments *Comprehension* we used two data models, i.e., ADAMS and EasyClinic (see Table 2). Concerning the ADAMS system we extracted the sub-diagram to manage the artefact versions and the teams working on the artefacts (in the following we denote it with *ADAMS-TeamManagement*). With regard to the EasyClinic data set, we extracted the sub-diagram to manage the patients' visits and bookings (in the following we denote it with *EasyClinic-BookingManagement*). The data models of the systems *Company* and *University* were used in the set of controlled experiments *Maintenance*. Finally, in the set of controlled experiments *Verification* we also used the data models of ADAMS and EasyClinic (see Table 2). Concerning the ADAMS data model, we extracted another sub-diagram describing the management of the files associated to an artefact and the event subscriptions (in the following we denote it with *ADAMS-EventManagement*). With regard to the EasyClinic system, we extracted the sub-diagram describing the management of the visits made by the doctors and the medical exams associated to a visit (in the following we denote it with *EasyClinic-VisitManagement*).

Because the maintenance activities require a sufficient knowledge of the system domain to be performed, we selected two software systems, i.e., *Company* and *University*, well known by subjects. The data models of these software systems are used as examples in the Databases and Software Engineering books used at the University of Salerno. For these systems we have the data models represented by ER and UML class diagrams. Concerning the EasyClinic system, an ER diagram represents the original data model. To obtain a UML class diagram representing the same data model, one of the author translated the original ER diagram. To mitigate translation errors, another author validated the translated data model. The same approach was also used to translate the original data model of ADAMS from an UML class diagram to an ER diagram. Thus, we can consider consistent and complete the UML class diagrams with respect to the ER diagrams.

2.3 Hypothesis Formulation

The main objective of our study was to analyse whether UML class diagrams are more comprehensible than ER diagrams during comprehension, maintenance, and verification activities on data models. Thus, the null-hypotheses were formulated for testing the effect of the design notation (i.e., ER or UML diagrams) on the comprehensibility, maintainability, and verifiability of a data model:

- H_{0_c} : there is no difference between the support provided by ER and UML class diagrams when performing comprehension tasks on data models (Comprehension Support).
- H_{0_m} : there is no difference between the support provided by ER and UML class diagrams when identifying the change to perform on data models to meet a maintenance request (Maintenance Support).
- H_{0_v} : there is no difference between the support provided by ER and UML class diagrams when identifying the defects in data models during verification activities (Verification Support).

The three hypotheses are summarised in Table 3, along with their alternative hypotheses. When the null hypothesis can be rejected with relatively high confidence

Table 3 Formal definitions of the experiment hypotheses

Experiment	Hypothesis	
Comprehension	Null hypothesis	$H_{0c} : \text{ComprehensionSupport}(UML) = \text{ComprehensionSupport}(ER)$
	Alternative hypothesis	$H_{ac} : \text{ComprehensionSupport}(UML) > \text{ComprehensionSupport}(ER)$
Maintenance	Null hypothesis	$H_{0m} : \text{MaintenanceSupport}(UML) = \text{MaintenanceSupport}(ER)$
	Alternative hypothesis	$H_{am} : \text{MaintenanceSupport}(UML) > \text{MaintenanceSupport}(ER)$
Verification	Null hypothesis	$H_{0v} : \text{VerificationSupport}(UML) = \text{VerificationSupport}(ER)$
	Alternative hypothesis	$H_{av} : \text{VerificationSupport}(UML) > \text{VerificationSupport}(ER)$

it is possible to accept an alternative hypothesis, which admits a positive effect of one of the two notations on the comprehensibility/maintainability/verifiability of the data model. In our experiment, we decided to accept alternative hypotheses that admit better performances of subjects when performing the assigned tasks on data models represented by UML class diagrams (see Table 3). Note that H_{0c} was tested in the set of controlled experiments *Comprehension*, while H_{0m} and H_{0v} were tested in the sets of controlled experiments *Maintenance* and *Verification*, respectively.

2.4 Identification and Definition of the Main Factor and the Co-factors

We performed a single factor experiment, where the main factor is represented by the design notation used to represent a data model. This factor is denoted as **Method** and has two levels, i.e., ER diagram (*ER*) or UML class diagram (*UML*). However, to better assess the effect of **Method** it was necessary to control other factors (called co-factors) that may impact the results achieved by the subjects and be confounded with the effect of the main factor. In the context of our study, we identify the following co-factors:

- **Ability**: the subjects' ability. A quantitative assessment of the ability level of 2nd year B.Sc. and 1st year M.Sc. students was obtained by considering the average grades obtained at the previous exams of Software Engineering and Databases. In particular, students with average grades below a fixed threshold, i.e., 24/30³ were classified as *Low Ability (Low)*, while the other students were classified as *High Ability (High)*. It is important to note that other criteria could be used to assess the subjects' ability. However, the focus of our experiments is on notations for data modelling, thus considering the grades achieved by the students in courses on this topic should be the best alternative. As for fresher bachelor students we considered the grade obtained at the High School diploma.

³We decided to select such a threshold as it represents the median of the possible grades for any exam to be passed by a student in an Italian University (min 18/30 and max 30/30).

In particular, students that achieved a grade lower than 80/100⁴ were classified as *Low Ability (Low)*, while the remaining ones as *High Ability (High)*.

- **ER and UML experience:** in the context of our experiments we had three different populations of subjects, i.e., fresher, bachelor, and master students. Fresher students had no experience in software design and development, while 2nd year B.Sc. and 1st year M.Sc. students had knowledge of both software development and software documentation, including database design and documentation. Moreover, fresher students did not know the ER and UML diagrams, while bachelor and master students had a fairly good knowledge of these notations and master students were more trained than bachelor students on the design methods. We were also interested in analysing the effect of the ER and UML experience since the different levels of education (and, consequently, the different levels of UML and ER experience) may impact the results achieved by subjects.
- **System:** in the context of the experiment students had to perform a comprehension/maintenance/verification activity on two different systems (see Section 2.2.2). Even if we tried to select software systems of a comparable size and tried to balance the complexity of the data models by using as heuristic the Miller's rule, there is still the risk that the system complexity may have a confounding effect with **Method**. For this reason we also considered the modelled system as an experimental co-factors.
- **Lab:** the experiments were organised in two laboratory sessions (see Section 2.6. In the first session subjects performed the task using UML (or ER) and in the other session they performed the task using ER (or UML). Although the experimental design limits the learning effect, it is still important to analyse whether subjects perform differently across subsequent labs.

2.5 Measurement of the Dependent Variables

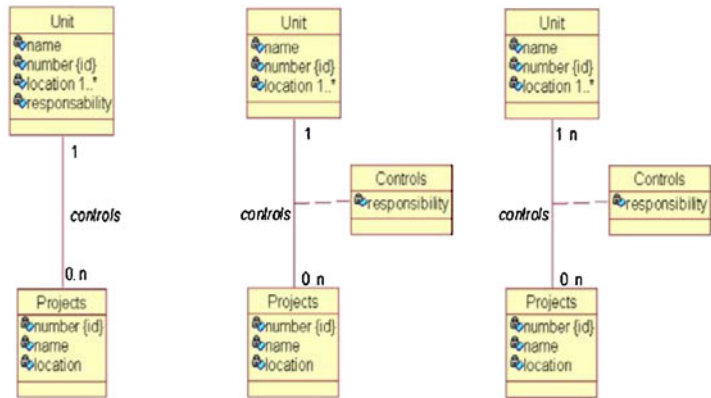
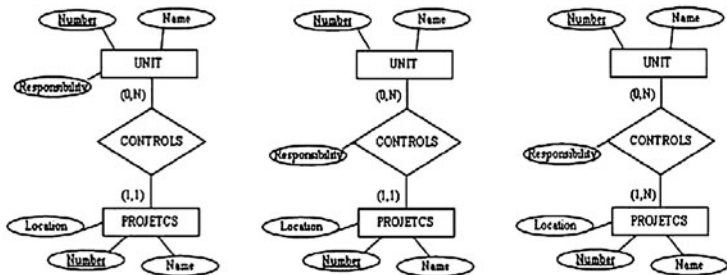
The main outcome observed in the set of controlled experiments *Comprehension* was the comprehension level. To evaluate it, we asked the subjects to answer a questionnaire (similar to Kuzniarz et al. 2004; Purchase et al. 2001, 2004; Ricca et al. 2007) consisting of 5 multiple choice questions where each question has one or more correct answers (see Table 4 for sample questions and De Lucia et al. 2008a for the complete questionnaire). The number of answers is the same for each question (i.e., three answers), while the number of correct answers is different. The questions cover all the entities and the relationships of the data model presented to the subjects.

For the set of controlled experiments *Maintenance* the main outcome observed is the number of correct identified changes. To evaluate it we adopted a similar approach as the assessment of the comprehension level. In particular, we asked the subjects to answer a questionnaire consisting of 5 multiple choice questions (see Table 4 for sample questions and De Lucia et al. 2008a for the complete questionnaire). The number of answers is the same for each question (i.e., three answers), but in this case each question has only one correct answers. In each question a change

⁴Also in this case we decided to select such a threshold as it represents the median of the possible grades (min 60/100 and max 100/100).

Table 4 Sample of comprehension, maintenance, and verification tasks

Type	Task
Comprehension	1. Consider the relationship between Artefact and Version and select the correct statements: ☐ An artefact can have more versions ☐ An artefact can have at most one version ☐ An artefact could have no versions 2. Consider the class Resource and its relationships. Which are the correct statements: ☐ Skills of are source can be identified ☐ It is possible to verify if a resource is allocated on an artefact ☐ It is possible to identify the telephone number of a resource
Maintenance	1. We want to keep track of the project responsibility of each unit. A unit can control more than one projects and can have different responsibilities in different projects. Each project is controlled by at least one unit. What are the modifications to be accomplished to carry out this request? Select the correct UML class diagram (or ER diagram).

Class diagrams**ER diagrams**

Verification	Analyse the requirements of the system and highlight in the data model the points that do not satisfy the requirements 1. An artefact... – is characterised by a name, a description, a status, a creation data, a starting and ending date; – has a type. The artefact type is characterised by a name, a description, an icon, and an identifier. Moreover, almost a file type has to be specified for each artefact type. Clearly, the same file type (characterised by a name, a description and an extension) can be associated to one or more artefact types; – should depend by other artefacts (<i>target</i>). In this case, it is necessary to specify the direction as well as the stereotype of the dependence; – should be composed of one or more artefacts. In this case, for each component artefacts (<i>child</i>) it is necessary to specify its position in the hierarchy; – has at least one resource allocated on it. Each resource allocated on an artefacts has the possibility to subscribe an event on it specifying the subscription level (<i>eventLevel</i>).
--------------	--

request is presented and a reader has to select which are the correct changes to perform on the data model to meet the change request.

The structure of the questionnaires allowed us to assess the answers using well known Information Retrieval metrics, namely recall and precision (Baeza-Yates and Ribeiro-Neto 1999). Indeed, since the questionnaire is composed of multiple-choice questions, we could compute recall and precision for each question as follow:

$$recall_{s,i} = \frac{|answer_{s,i} \cap correct_i|}{|correct_i|} \%$$

$$precision_{s,i} = \frac{|answer_{s,i} \cap correct_i|}{|answer_{s,i}|} \%$$

where $answer_{s,i}$ is the set of answers given by subject s to question i and $correct_i$ is the set of correct answers expected for question i . As we can see, it is not possible to calculate the precision for a question when no answer is provided. For this reason, we used the following aggregate metrics, rather than the average precision and recall (Antoniol et al. 2002; Zimmermann et al. 2005):

$$recall_s = \frac{\sum_i |answer_{s,i} \cap correct_i|}{\sum_i |correct_i|} \%$$

$$precision_s = \frac{\sum_i |answer_{s,i} \cap correct_i|}{\sum_i |answer_{s,i}|} \%$$

With regard to the outcome observed in the set of controlled experiments *Verification*, i.e., number of errors identified by software engineers during the verification activities, we evaluated it by presenting to the subjects a list of requirements and asking them to highlight the points in the data models that did not satisfy the requirements (see Table 4 for sample questions and De Lucia et al. 2008a for the complete questionnaire). Therefore, also in this case recall and precision (Baeza-Yates and Ribeiro-Neto 1999) were used to evaluate the results achieved by a subject:

$$recall_s = \frac{|highlightedmismatch_s \cap mismatch|}{|correctmismatch|} \%$$

$$precision_s = \frac{|highlightedmismatch_s \cap mismatch|}{|highlightedmismatch_s|} \%$$

where $highlighted\ mismatch_s$ is the set of mismatch identified by subject s and $correct\ mismatch$ is the number of actual mismatch contained in the diagram, with respect to the requirements.

It is worth noting that recall and precision measure two different concepts. Thus, we decided to use an aggregate measure (i.e., F-measure (Baeza-Yates and Ribeiro-Neto 1999)) to obtain a balance between them:

$$F-measure_s = 2 * \frac{precision_s * recall_s}{precision_s + recall_s} \%$$

To summarise, the dependent variables of the three sets of experiments, i.e., *Comprehension*, *Maintenance*, and *Verification*, were **Comprehension Support**, **Maintenance Support**, and **Verification Support**, respectively. They were calculated as the F-measure of the aggregate precision and recall achieved by each subject.

After the execution of each experiment, we collected the questionnaires filled-in by each subject in each laboratory session. The results of the questionnaire were reported by one of the author in spreadsheets aiming at performing the data analysis. To reduce the risk of human errors in reporting the results, another author double-checked the inserted data. Once the data were validated, the F-measures achieved by the subject were calculated.

2.6 Experimental Procedure

Each experiment was organised in two laboratory sessions. In particular, in the context of the experiment subjects had to perform two comprehension activities (or maintenance or verification activities, according to the aim of the experiment) on the data models of two different software systems. Each subject performed an activity on an UML diagram (or ER diagram) in one laboratory session and exploited an ER diagram (or UML diagram) to perform the same activity on the data model of a different software system in the second laboratory session. Note that between the two laboratory sessions we gave the subjects a break of 10 min. Leaving the laboratory was not allowed during the break and the experimenters also monitored the students to avoid collaboration and communication between them.

The organisation of each group of subjects in each experimental lab (*Lab1* and *Lab2*) followed the design shown in Table 5. In particular, the rows represent the four experimental groups, whereas the columns refers to the design notation used to represent the data model (i.e., ER and UML). The table cells show, for each group and design notation employed, which system students worked on, indicated with *S1* and *S2*, and the laboratory session, indicated with *Lab1* and *Lab2*. Such an experimental design ensured that each subject worked on different systems in the two laboratory sessions, using each time a different design method. The chosen design also permitted to consider different combinations of **System** and **Method** in different order across laboratory sessions. It is important to note that all the experiments and the replications followed the same design (see Table 5). The only differences across the different sets of experiments are represented by the assigned task and the two systems, i.e., *S1* and *S2*.

Table 5 Experimental design

Group	Treatment	
	ER	UML
A	S1, Lab1	S2, Lab2
B	S2, Lab2	S1, Lab1
C	S2, Lab1	S1, Lab2
D	S1, Lab2	S2, Lab1

Comprehension: S1 = ADAMS-TeamManagement, S2 = EasyClinic-BookingManagement

Maintenance: S1 = Company, S2 = University

Verification: S1 = ADAMS-EventManagement, S2 = EasyClinic-VisitManagement

S1 and S2 indicate the systems used in the experimentation

Subjects performed the assigned task individually. We also wanted to account for individual differences in such a way as to increase the statistical power of our analysis and ensure that the design would not create any systematic bias in our results (Henderson 2003). In particular, we observed that the selected students had different levels of ability. This was not a problem, since the variations of ability are also present among professionals and all levels of ability should therefore be represented. To simulate a real industrial context, we grouped students in two blocks according to their ability level (*High* and *Low*) and they were randomly distributed among the laboratory groups, making sure that *High* and *Low* ability subjects were equally distributed across groups. This was necessary to balance the number of high and low ability subjects between the laboratory groups. In this way each combination of design notation and ability level (e.g., *UML* and *High* ability) has an equal number of observations (subjects). Thus, according to Wohlin et al. (2000) we exploited a “randomised block design”. However, we also considered the subjects ability as a co-factor in the analysis for two reasons:

- The number of *High* and *Low* ability subjects was not the same.
- We were also interested in analysing/comparing the performances of subjects with different abilities when performing comprehension, verification, and maintenance activities on data models represented by ER and UML class diagrams.

The latter point is particularly useful for a business analyst point of view. Indeed, the results of such an analysis might be useful to evaluate the possibility of adopting a design method within a software project, depending on the skills of the involved human resources.

Before the experiments, subjects were trained on both ER and UML class diagrams. To avoid bias (i) the training was performed on a data model not related to the systems selected for the experimentation and (ii) its duration was exactly the same for the experiment and the replications. Right before the experiments, we also showed to the students a presentation with detailed instructions related to the tasks to be performed.

According to the experimental design, each subject was involved in two laboratory sessions, where subjects had a fixed amount of time to complete the required tasks. In particular, for the maintenance and verification activities students had 30 min to complete the assigned task, while 15 min were given to complete the task for the comprehension activities. Each laboratory consists of specific activities (according to the goal of the specific controlled experiment) on the data models of the assigned systems documented either by ER or UML class diagrams.

After each laboratory session, subjects filled-in a survey questionnaire. It was composed of four questions (Q1–Q4 in Table 6) expecting closed answers according to the Likert scale (Oppenheim 1992)—from 1 (strongly agree) to 5 (strongly disagree)—to assess if the task and its objectives were clear, if subjects had enough time, as well as to identify if students experienced difficulties performing the tasks. A further question (Q5 in Table 6) was also asked to the subjects involved in the sets of controlled experiments *Maintenance* and *Verification*.

Moreover, after the experiment, the subjects expressed their personal opinion about what was the notation that better supported them in performing the assigned tasks (see Q6 in Table 6). Finally, all the subjects, except the fresher bachelor students, also indicated their experience level on the two notations, by selecting a

Table 6 Post-experiment questionnaire

Id	Experiment	Question
Q1	All	I had enough time to perform the lab task
Q2	All	The objectives of the lab were perfectly clear to me
Q3	All	The task I had to perform was perfectly clear to me
Q4	All	I experienced no major difficulties in performing the task
Q5	Maintenance and Verification	The application domain of the system is clear
Q6	All	Which was the easier notation to comprehend/ maintain/verify?
Q7	All except Comprehension with fresher B.Sc. students	Evaluate your experience level on UML class diagrams
Q8	All except Comprehension with fresher B.Sc. students	Evaluate your experience level on ER diagrams

value from 1—very high experience—to 5—very low experience (see questions Q7 and Q8 in Table 6).

2.7 Data Analysis

Since experiments were organised as longitudinal studies,⁵ where each subject performed a task on two different models (i.e., *S1* or *S2*) with the two possible treatments (i.e., *ER* or *UML*), it was possible to use a paired Wilcoxon one-tailed test (Conover 1998) to analyse the differences exhibited by each subject for the two treatments. A one-tailed paired t-test (Conover 1998) can be used as alternative to the Wilcoxon test. However, we decided to use the Wilcoxon test since it is resilient to strong departures from the t-test assumptions (Briand et al. 2005). The achieved results were intended as statistically significant at $\alpha = 0.05$. This means that if the derived p-value is less than 0.05, it can be concluded that the null hypothesis can be rejected. Thus, according to our hypotheses formulation we can deduce that *UML* has a significant positive effect on the dependent variable.

The chosen design also permitted to analyse the effects of co-factors and their interaction with the main factor. However, as highlighted by Kitchenham et al. (2003) there are several difficulties to analyse cross-over designs such as the design used in our controlled experiments. For this reason, to not violate any model assumptions we only performed an informal analysis of the effect of co-factors on the dependent variables through interaction plots (Devore and Farnum 1999). Interaction plots are simple line graphs where the means on the dependent variable for each level of one factor are plotted over all the levels of the second factor. The resulting lines are parallel when there is no interaction and nonparallel when interaction is present. The interaction plots was also used to analyse for each set of controlled experiments the influence of subjects' experience on the dependent variable. To perform such an analysis we employed as data set the set of all the subjects, similarly to Wohlin et al. (2000) and Ricca et al. (2007). This was possible because for each set of experiments

⁵ A longitudinal study is a research study conducted over a period of time (Wohlin et al. 2000). In our case, each controlled experiment is a longitudinal study because it was organised in two laboratory sessions.

the design, material, and procedure was exactly the same. Indeed, the only difference among students involved in the experiments was the ER and UML experience and it was considered as an experimental co-factor.

We also decided to not perform a detailed analysis of the co-factors to not confuse the main message of our experimentation. In particular, we were interested in analysing whether or not UML class diagrams are more comprehensible than ER diagrams during different kinds of activities on data models. As explained above, such an analysis was performed by using a Wilcoxon one-tailed test.

Finally, to better understand the experimental results, we also analysed the feedback provided by each subject filling in the post-experiment questionnaire after each laboratory session. To this end, we analysed boxplots of answers and tested statistical significance of differences using a Mann-Whitney one-tailed test (Conover 1998). This investigation was carried out to capture the students' point of view about the two employed notations.

3 Experimental Results

In the next sections we report on the results achieved in the three sets of controlled experiments, i.e., *Comprehension*, *Maintenance*, and *Verification*, as well as the results achieved in the survey questionnaires.

3.1 ER vs UML during Comprehension Activities

Tables 7, 8, and 9 show descriptive statistics (i.e., median, mean, and standard deviation)—grouped by **Method** and **System**—for the dependent variable **Comprehension Support** of the *Comprehension* experiments carried out with fresher students (Comp-Fsc), with bachelor students (Comp-Bsc), and with master students (Comp-Msc).

Next subsections report the statistical analysis of the results achieved in the three experiments by assessing the effect on the dependent variable of the main factor and by analysing the effect of the co-factors.

3.1.1 Influence of Method

To test the hypothesis H_0 , we analysed the effect of **Method** on **Comprehension Support**. Table 10 reports the p-values of the Wilcoxon tests. The table also reports descriptive statistics of differences achieved by subjects. In particular, for each subject we calculated the difference between the result achieved by the subject performing the task using the UML class diagram and the result achieved by the

Table 7 Experiment Comp-Fsc: descriptive statistics of Comprehension Support by method and system

System	ER			UML		
	Median	Mean	Std. Dev.	Median	Mean	Std. Dev.
All	71.00	68.35	20.81	76.00	75.65	14.45
S1	71.00	67.24	19.42	73.00	75.00	14.63
S2	71.00	69.47	22.36	77.00	76.29	14.45

Table 8 Experiment Comp-Bsc: descriptive statistics of Comprehension Support by method and system

System	ER			UML		
	Median	Mean	Std. Dev.	Median	Mean	Std. Dev.
All	78.46	73.29	20.79	85.71	82.48	13.75
S1	75.96	70.79	20.95	85.71	82.65	11.29
S2	80.00	75.79	20.87	86.61	82.31	16.13

same subject performing the task using the ER diagram. The statistics (i.e., mean, median, and standard deviation) reported in the table describe the distribution of the differences achieved by subjects. We also reported the percentage of subjects that achieved better results performing the comprehension activity using the UML class diagram (column % of of Pos. Effect).

The results revealed that the null hypothesis H_{0c} can be rejected in all the experiments and also when considering the set of all subjects. This means that the use of UML class diagrams significantly affects the comprehension level achieved by the subjects, as also confirmed by the fact that the mean differences are positives in all the experiments. Indeed, about 60% of the subjects involved in the three controlled experiments achieved better results when an UML class diagram represents the data model of the system.

3.1.2 Analysis of Co-factors

The analysis of the interaction plots in Fig. 1a, b, and c highlights that subjects with *High* ability achieved better results than *Low* ability subjects in each experiment. Moreover, subjects using the UML class diagram achieved higher comprehension levels than subjects using the ER diagram. This result further supports the rejection of the null hypothesis H_{0c} .

As well as for **Ability**, the analysis of the interaction plot by **Method** and **Experience** (see Fig. 1d) reveals that bachelor students achieved better results than fresher students, while master students achieved better results than bachelor students. Moreover, all the students (independently from their UML and ER experience) achieved better comprehension levels with the UML class diagram.

Finally, the analysis of the interaction plots revealed no considerable effect of **System** and **Lab**, as well as no considerable interaction between **Method** and **System** and **Method** and **Lab** in the main experiment and in its replications.

Table 9 Experiment Comp-Msc: descriptive statistics of Comprehension Support by method and system

System	ER			UML		
	Median	Mean	Std. Dev.	Median	Mean	Std. Dev.
All	87.50	82.10	17.03	93.33	90.53	10.09
S1	87.50	82.21	19.02	90.42	86.68	11.15
S2	86.61	82.02	15.72	96.67	94.93	6.68

Table 10 Comprehension experiments: Wilcoxon tests and descriptive statistics of differences (by subject)

Experiment	#subj.	#obs.	Med.	Mean	Std. Dev.	p-value	% of Pos. effect
Comp-Fsc	68	136	7.00	7.29	27.08	0.028	61.76
Comp-Bsc	40	80	7.95	9.19	25.00	0.015	65.00
Comp-Msc	30	60	2.92	8.43	18.76	0.021	70.00
All	138	276	6.50	8.09	24.73	2.42E-04	58.70

3.2 ER vs UML during Maintenance Activities

Tables 11 and 12 show the descriptive statistics (i.e., median, mean, and standard deviation) for the dependent variable **Maintenance Support**—grouped by **Method** and **System**—of the *Maintenance* experiments carried out with bachelor students (Maint-Bsc) and with master students (Maint-Msc).

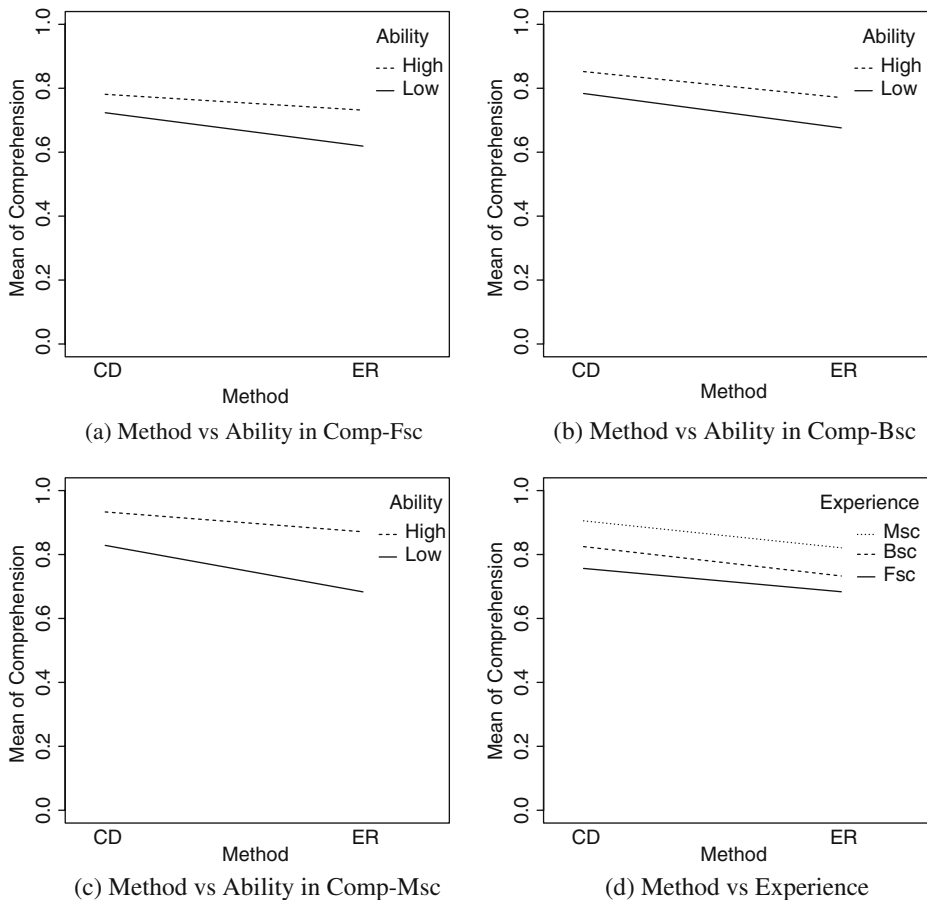
**Fig. 1** Analysis of co-factors on comprehension through interaction plots

Table 11 Experiment Maint-Bsc: descriptive statistics of Maintenance Support by method and system

System	ER			UML		
	Median	Mean	Std. Dev.	Median	Mean	Std. Dev.
All	80.00	67.25	22.64	60.00	65.02	21.12
S1	80.00	64.50	21.64	60.00	62.22	19.11
S2	70.00	70.00	23.84	70.00	67.82	23.10

In the following we describe the results achieved in the two experiments, assessing the effect on the dependent variable of the main factor and by analysing the effect of co-factors.

3.2.1 Influence of Method

To test the hypothesis H_{0_m} , we analysed the effect of **Method** on **Maintenance Support**. The p-values are reported in Table 13, together with the descriptive statistics of the differences, which are obtained subtracting the number of correct identified changes achieved by each subject with the ER diagram from the number of correct identified changes he/she achieved with the UML class diagram. Moreover, the table shows the percentage of subjects that achieved better results using the UML class diagram (column % of Pos. Effect of Table 13).

The results revealed that the null hypothesis H_{0_m} cannot be rejected in both the experiments and when considering all subjects. This means that there is not statistically significant difference between the number of correct identified changes achieved with ER and UML class diagrams. However, more than 47% of subjects involved in both the experiments achieved better results using ER diagrams and about 18% achieved the same results with both notations. This means that only 35% of the subjects achieved better results when performing the assigned task with the UML class diagram.

3.2.2 Analysis of Co-factors

The analysis of the interaction plots in Fig. 2a and b highlights that subjects with *High* ability achieved better results than *Low* ability subjects. Furthermore, we can observe that when performing the maintenance activities with the UML class diagram *High* ability master students achieved better results than master students with *Low* ability, while the same difference was not found when the task is performed with the ER diagram.

We also analysed the influence of the experience by using the interaction plots (see Fig. 2c). The analysis revealed that there is not interaction between **Method**

Table 12 Experiment Maint-Msc: descriptive statistics of Maintenance Support by method and system

System	ER			UML		
	Median	Mean	Std. Dev.	Median	Mean	Std. Dev.
All	80.00	73.18	24.90	70.00	68.26	24.57
S1	70.00	67.14	25.55	60.00	66.46	23.45
S2	85.45	78.46	23.87	80.00	70.32	26.53

Table 13 Maintenance experiments: Wilcoxon tests and descriptive statistics of differences (by subject)

Experiment	#subj.	#obj.	Median	Mean	Std. Dev.	p-value	% of Pos. effect
Maint-Bsc	40	80	0.00	−2.23	31.52	0.706	37.50
Maint-Msc	30	60	0.00	−4.92	38.12	0.731	26.67
All	70	140	0.00	−3.38	34.27	0.799	35.71

and **Experience** and that both **Method** and **Experience** did not influence the results. In particular, bachelor and master students achieved comparable results. Moreover, differently from the other set of experiments, both master and bachelor students achieved slightly better results with the ER diagram.

As in the case of *Comprehension* experiments, interaction plots revealed no effect of **System** and **Lab**, as well as no interaction between **Method** and **System** and **Method** and **Lab** in both the experiments.

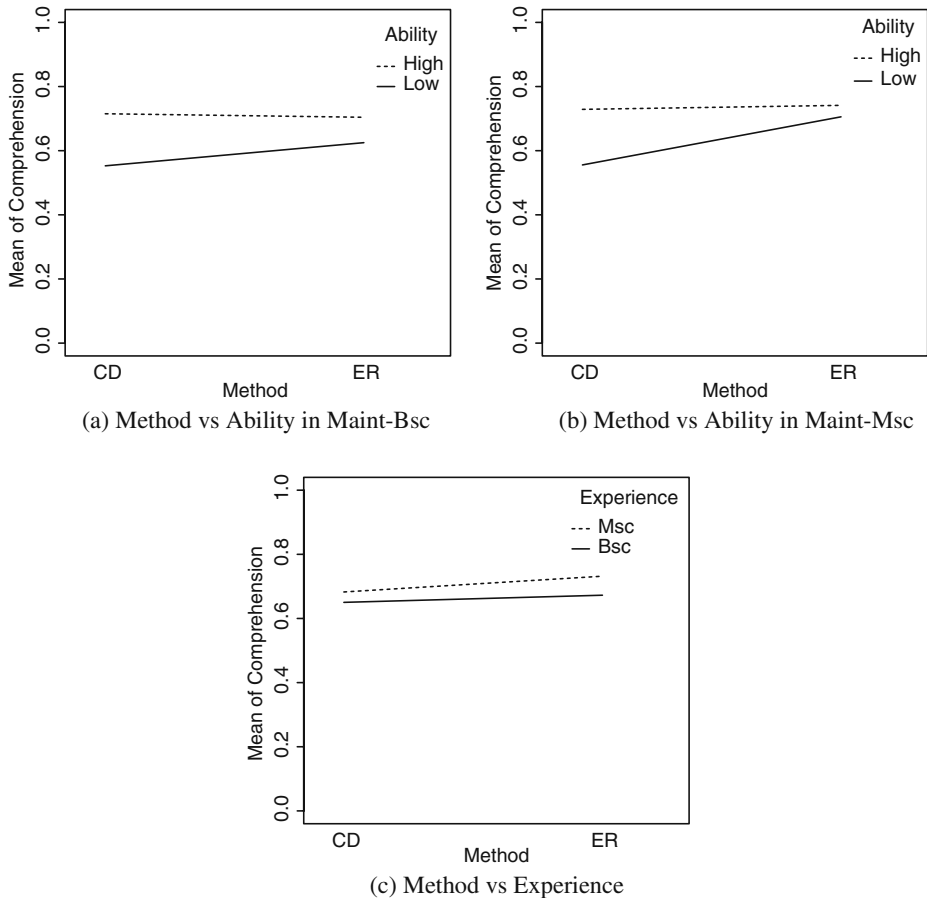
**Fig. 2** Analysis of co-factors on maintenance support through interaction plots

Table 14 Experiment Verif-Bsc: descriptive statistics of Verification Support by method and system

System	ER			UML		
	Med.	Mean	Std. Dev.	Med.	Mean	Std. Dev.
All	50.00	46.57	20.11	60.18	57.57	20.07
S1	36.36	41.75	20.35	60.66	58.71	20.76
S2	53.33	52.47	18.68	59.34	56.64	19.92

3.3 ER vs UML during Verification Activities

Tables 14 and 15 show descriptive statistics (i.e., median, mean, and standard deviation)—grouped by **Method** and **System**—for the dependent variable **Verification Support** of the *Verification* experiments carried out with bachelor students (Verif-Bsc) and with master students (Verif-Msc).

Next subsections report the statistical analysis of the results, by assessing the effect on the dependent variable of the main factor and by analysing the effects of the co-factors.

3.3.1 Influence of Method

To test the hypothesis H_{0_1} , we analysed the effect of **Method** on **Verification Support**. Table 16 reports the p-values of the Wilcoxon tests, the descriptive statistics of differences achieved by subjects, and the percentage of positive differences (i.e., % of Pos. Effect), that is the percentage of subjects that achieved better results when performing the verification activities with the UML class diagram.

The achieved results revealed that the null hypothesis H_{0_1} can be rejected for experiment Verif-Bsc and considering all the subjects involved in the experiment and its replication. This means that the use of the UML class diagram significantly affected the number of errors identified by the bachelor students.

3.3.2 Analysis of Co-factors

The interaction plots in Fig. 3a and b show that subjects with *High* ability obtained better results than *Low* ability subjects in both the experiments. Also in this case, subjects achieved better performances with the UML class diagram.

We also analysed the influence of the subjects' experience on the results (see Fig. 3c). The interaction plot shows no interaction between **Method** and **Experience** and that master students achieved better results than bachelor students. However,

Table 15 Experiment Verif-Msc: descriptive statistics of Verification Support by method and system

System	ER			UML		
	Med.	Mean	Std. Dev.	Med.	Mean	Std. Dev.
All	57.14	57.77	24.13	60.18	60.31	22.49
S1	57.14	61.04	24.13	69.05	65.82	19.97
S2	56.35	54.90	24.53	51.67	54.02	24.25

Table 16 Verification experiments: Wilcoxon tests and descriptive statistics of differences (by subject)

Experiment	#subj.	#obs.	Median	Mean	Std. Dev.	p-value	% of Pos. effect
Verif-Bsc	40	80	11.15	11.00	25.07	0.007	65.00
Verif-Msc	30	60	−5.46	2.55	33.89	0.451	40.00
All	70	140	4.95	7.38	29.52	0.025	50.29

such a difference is mitigated by the use of the UML class diagram. In particular, when using the UML class diagram master and bachelor students obtained almost the same results.

The analysis of the interaction plot by **Method** and **System** and **Method** and **Lab** revealed no effect of the co-factors as well as no considerable interaction between the co-factors and the main factor.

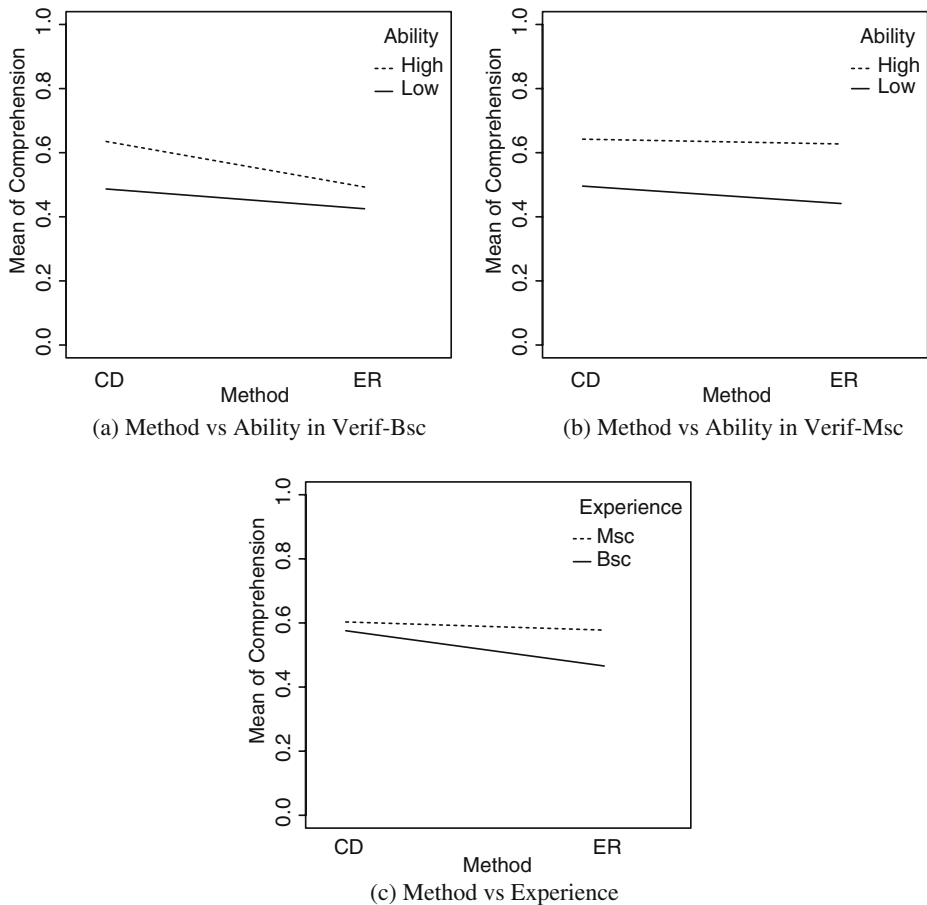
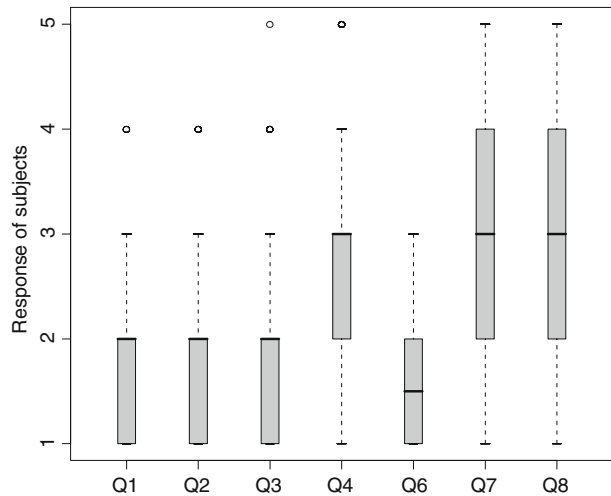
**Fig. 3** Analysis of co-factors on verification support through interaction plots

Fig. 4 Comprehension experiments: answers of subjects



3.4 Analysis of Subjects' Feedback

Other than the objective analysis presented in the previous sections, we carried out a subjective analysis by exploiting the feedbacks provided by each subject filling in the post-experiment questionnaire. This investigation was carried out to capture the students' point of view about the performed task and the two employed notations.

Figures 4, 5, 6 show boxplots of answers for each set of experiments. The analysis of the answers suggested that subjects had enough time to carry out the task (Q1). However, as expected in each set of experiments graduate students had a greater perception that the time was enough than undergraduate students ($p\text{-value} = 0.010$ for *Comprehension*, $p\text{-value} = 0.031$ for *Maintenance*, and $p\text{-value} = 0.027$ for

Fig. 5 Maintenance experiments: answers of subjects

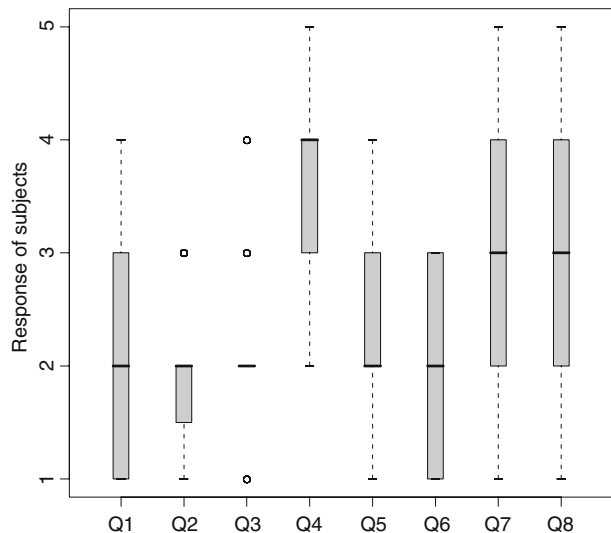
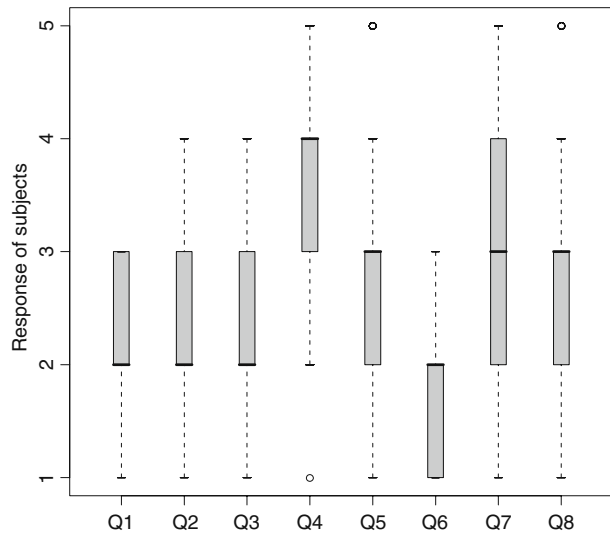


Fig. 6 Verification experiments: answers of subjects



Verification). Differently, no significant difference was found between answers provided by subjects with *High* and *Low* ability (p-value = 0.754 for *Comprehension*, p-value = 0.656 for *Maintenance*, and p-value = 0.555 for *Verification*). Nevertheless, the analysis also revealed that when considering only the experiments with graduate students there was significant difference between answers of subjects with *High* and *Low* ability (p-value = 0.007 for *Comprehension*, p-value = 0.009 for *Maintenance*, and p-value = 0.003 for *Verification*).

By looking at the answers to questions Q2 and Q3, we found that the objectives and the laboratory task to perform were clear to the subjects. Moreover, as expected graduate students felt the objectives and the laboratory task to perform significantly clearer than undergraduate students (p-value = 0.001 for *Comprehension*, p-value = 0.011 for *Maintenance*, and p-value = 0.017 for *Verification* in the case of Q2 and p-value = 0.002 for *Comprehension*, p-value = 0.007 for *Maintenance*, and p-value = 0.005 for *Verification* in the case of Q3), while no significant differences emerged between subjects with *High* and *Low* ability (p-value = 0.565 for *Comprehension*, p-value = 0.411 for *Maintenance*, and p-value = 0.317 for *Verification*).

The subjects experienced particular difficulties to perform the maintenance and verification activities (Q4). In particular, the median of the answers is 4 (i.e., I disagree) for graduate and undergraduate and *High* and *Low* ability subjects. As in the case of questions Q1, Q2, and Q3 no significant difference was found between the answers of *Low* and *High* ability subjects (p-value = 0.654 for *Comprehension*, p-value = 0.714 for *Maintenance*, and p-value = 0.327 for *Verification*). Differently from the other questions, no significant difference was found between the answers of undergraduate and graduate students (p-value = 0.371 for *Comprehension*, p-value = 0.523 for *Maintenance*, and p-value = 0.437 for *Verification*). It is important to note that for fresher students the median of answers is 3 (i.e., a mean difficulty).

Concerning the system domain knowledge (i.e., question Q5), the results showed that during the experiments subjects had a perception of an acceptable domain knowledge. No significant difference was found between *High* and *Low* ability

subjects (p -value = 0.565 for *Maintenance* and p -value = 0.337 for *Verification*) while, as expected, graduate students declared to have a greater system domain knowledge than undergraduate students (p -value = 0.002 for *Maintenance* and p -value = 0.012 for *Verification*).

With regard to the easier notation to comprehend, the median of the answers to question Q6 provided by bachelor and master students is “No difference” (i.e., in experiments Comp-Bsc and Comp-Msc). However, the subjects preferring UML class diagrams were 31 (18 were undergraduate and 13 graduate, 21 with *High* ability and 10 with *Low* ability) while the subjects preferring ER diagrams were 18 (9 were undergraduate and 9 graduate, 14 with *High* ability and 4 with *Low* ability). The fresher students answered in a different way to this question. In particular, the fresher students preferring ER diagrams were 38 (16 with *High* ability and 22 with *Low* ability) while the fresher students preferring UML class diagrams were 30 (23 with *High* ability and 7 with *Low* ability). We can observe that high ability subjects preferred UML class diagrams even if they are fresher students.

As for the experiment Maint-Bsc and Maint-Msc, the median of the answers to the question on the easier notation to maintain (Q6) is also 2. The subjects preferring UML class diagrams were 21 (10 were undergraduate and 11 graduate, 11 with *High* ability and 10 with *Low* ability) while the subjects preferring ER diagrams were 23 (15 were undergraduate and 8 graduate, 17 with *High* ability and 6 with *Low* ability).

The median of answers to question Q6 in experiments Verif-Bsc and Verif-Msc (i.e., *Verification* experiments) is also 2. Also in this case, the number of subjects preferring UML class diagrams is greater than the number of subjects preferring ER diagrams. In particular, the subjects preferring UML class diagrams were 28 (16 were undergraduate and 12 graduate, 19 with *High* ability and 9 with *Low* ability) while the subjects preferring ER diagrams were 13 (9 were undergraduate and 4 graduate, 10 with *High* ability and 3 with *Low* ability).

The answers to questions Q7 and Q8 reveal that subjects declared to have a medium experience with ER and UML class diagrams. In particular, the median of the answers is 3 for both questions. Furthermore, no significant difference was found between the answers of *High* and *Low* ability subjects (p -value = 0.766 for *Comprehension*, p -value = 0.824 for *Maintenance*, and p -value = 0.581 for *Verification*) and between the answers of undergraduate and graduate (p -value = 0.573 for *Comprehension*, p -value = 0.543 for *Maintenance*, and p -value = 0.217 for *Verification*).

4 Validity Evaluation

In this section we discuss the threats to validity that could affect our results, focusing the attention on *construct*, *internal*, *external*, and *conclusion* validity threats.

4.1 Internal Validity

Internal validity threats can be due to the learning effect experienced by subjects between labs. This was mitigated thanks to the experiment design: subjects worked, over the two labs, on different systems and using two different design methods (i.e., ER or UML class diagrams). Nevertheless, there was still the risk that, during

labs, subjects might have learned how to improve their results. However, the factor **Lab** was accounted as a factor in the analysis of the results to verify the absence of learning effect. Also the effect of the system complexity, represented by the confounding factor **System**, on the dependent variables was analysed to verify the balancing of the assigned data models.

Another concern relates to the fact that the subjects performed two subsequent activities, i.e., comprehension and verification, on the same system. To avoid learning effect we selected two independent and different sub-diagrams of the whole data model of the system.

As highlighted in Briand et al. (2005), one possible issue related to internal validity is due to the experiment design and concerns the possible information exchange among the subjects between the laboratories. To mitigate such a threat the experimenters monitored all the students during the experiment execution to avoid collaboration and communication between them.

Finally, a threat to validity is represented by the quality of teaching. In particular, if the UML teacher is good and the ER teacher is bad, probably students have more difficulties in performing the task with ER diagrams than using UML diagrams. For this reason, we analysed the students' evaluation on the quality of the courses of both Databases and Software Engineering. The analysis revealed no significant difference between the quality of the two courses. However, students are not the best people to assess the quality of teaching: they can assess the quality of their educational experience, but they had some difficulty to assess whether they have been taught a topic adequately. For this reason, replications of the study in other universities could further mitigate such a threat. Obviously, this threat could not affect the results achieved by fresher bachelor students.

4.2 External Validity

External validity threats concern the generalisation of the results and are always present when experimenting with students. Concerning the undergraduate and graduate students, they had an acceptable analysis, development, and programming experience, and they are not far from junior industrial analysts. In particular, in the context of the Software Engineering courses, both master and bachelor students had participated to software projects, where they experienced software development and documentation, including database design. Moreover, as highlighted by Arisholm and Sjöberg (2004) the difference between students and professionals is not always easy to identify. Nevertheless, there are several differences between industrial and academic contexts. For this reason, we plan to replicate the experiment with industrial subjects to corroborate the achieved findings. It is important to note that fresher bachelor students were involved in our experiments to investigate the behaviour of non-expert people during the requirement elicitation phase.

Finally, the size of the data models is small compared to industrial cases, but it is comparable with the size of models used in other related experimentations (see, for instance, Shoval and Shiran 1997; Briand et al. 2005; Ricca et al. 2007). Moreover, the selected data models are subdiagrams of more complex diagrams (we extracted these subdiagrams according to the Miller's rule (Miller 1956)). We decided to select data models with a relative small size since a controlled experiment requires that subjects complete the assigned task in a limited amount of time and without interruption

to keep variables under control. For this reason, we could not consider larger data models. Future work will be devoted to assess the usefulness of the notations on realistically sized artefacts. However, just the comparison of the two notations on small/medium artefacts is a worthy contribution.

4.3 Construct Validity

Construct validity threats that may be present in this experiment, i.e., interactions between different treatments, were mitigated by a proper design that allowed to separate the analysis of the different factors and of their interactions. In particular, in each set of experiments subjects worked on two different systems to avoid learning effects and to ensure that the differences in systems' complexity would not bias the results (we tried to select two data models of comparable complexity).

The method used to assess the comprehension support of a data model, the correctness of the implementation of change requests, and the detection error accuracy led to an easy measurement of the subjects' performance. In particular, the metric used to assess the subjects' performance is an aggregate measure of precision and recall that well reflects the results achieved by the subjects. We are also confident that the used tools (multiple-choice questions and defect identification) actually measure the comprehensibility (as well as the verifiability) of the data models. This is also confirmed by the fact that previous empirical studies also used similar approaches to measure the same attributes (see for instance Shoval and Shiran 1997; Shoval and Frumermann 1994; Bock and Ryan 1993; Palvia et al. 1992; Ricca et al. 2007). With regard to the maintainability we are aware that multiple-choice questions cannot actually measure the maintainability of a data model. Thus, different tools should have been used to measure such an attribute. However, maintenance activities are difficult tasks and required wide domain knowledge of the system to maintain. Unfortunately, our experimentation involved students that (i) did not have a wide knowledge of the modelled system (actually, it was not their project) and (ii) did not have a wide experience on the maintenance of data models. For this reason, the aim of our study was not to measure the maintainability of UML and ER diagrams, but to analyse whether UML class diagrams provide a better support than ER diagrams in the comprehension of the change to perform to meet a given request change. This is also an important activity as it is close to reviewing alternative solutions to a change request. Thus, to analyse such an aspect we decided to use a questionnaire where each question admitted only one correct answer. In particular, each question represents a change request and it includes one correct change to perform on the data model to meet the change request.

Since the assigned task had to be performed in a limited amount of time, the time pressure could represent another threat to validity. However, we decided the duration of each experiment taking into account previous laboratory exercises performed by the students involved in the experimentations during their courses. Furthermore, we also exploited our experience in performing similar controlled experiments in the past. However, all the subjects completed the assigned task and they declared (in the post-experiment questionnaires) to have enough time to complete the task. For these reasons we are confident that time pressure did not condition the results and thus we did not consider it as a confounding factor.

Concerning the selection of the subjects, we observed that the selected students had different levels of ability. It is important to note that the variations of ability are also present among professionals and all levels of ability should therefore be represented (Briand et al. 2005). Moreover, regarding the criteria used to obtain a quantitative evaluation of subjects' ability, we considered only two levels of ability (high and low) discriminating students taking into account the average grades they obtained at the previous exams for undergraduate and graduate students and the grade of the High School diploma in the case of fresher bachelor students. Clearly, more levels than low and high could have been used. Nevertheless, analyses performed with more levels did not yield any different or contrasting results.

We also used the level of education to measure the level of experience with ER and UML class diagrams. However, we also asked the subjects to rate their experience with UML class and ER diagrams in the final questionnaire. Thus, such a rate could be used to determine the level of experience with the experimented design notations. Clearly, the answers to the questionnaire were an auto-evaluation that students made with respect to their knowledge on the two notations. For this reason, these answers have mainly a subjective relevance (the positive or negative answer also depends on the self-esteem level and on the strong or weak personality of students) and for this reason we did not consider such information in our analysis. In general, both master and bachelor subjects declared to have a medium experience on the two notations, but master students are more trained on the experimented design notations. In particular, they had participated in real software projects during the internship made to conclude their bachelor university program. This is not only a conjecture since the results achieved by master students are generally better than the results achieved by bachelor students. For this reason, we decided to use a non-subjective parameter (i.e., the level of education) to evaluate the subject experience.

Another threat related to our population of subjects is that we represented customers by fresher bachelor students. If on one hand customers may have little knowledge about the design notations like fresher bachelor students, on the other hand customers have good knowledge about their business domains and rules. Thus, replicating the study with real customers is required to corroborate our findings.

To avoid social threats due to evaluation apprehension, students were not evaluated on the performances they achieved in the experiments. During the experiment, we monitored the subjects to verify whether they were motivated and made attention in performing the assigned task. We observed that students performed the required task with dedication and there was no abandonment. Moreover, students were aware that our goal was to evaluate the impact of using ER or UML class diagrams during modelling activities, but they were not aware of the exact hypotheses tested and of the particular dependent variables of interest.

It is important to note that the experiment has been carried out to evaluate the effectiveness of ER and UML class diagrams in supporting comprehension, verification, and maintenance of data models. Clearly, many other software engineering activities could have been considered, but those three were selected as they correspond to typical expectations from people using analysis and design notations (Briand et al. 2005). However, especially where the design of performance-critical, data-intensive software like databases is concerned, there are other more specific key considerations as well, e.g., analysability. For instance, one may choose to sacrifice expressiveness (and thus comprehensibility) for analysability or other properties. For

this reason, future work will be devoted to perform a deeper analysis on more specific properties of the two notations.

Finally, in our experiment we focused the attention on how the two notations (ER and UML class diagrams) represent entities, attributes, and relationships. However, replications are also needed to further compare ER and UML diagrams, taking into account other elements of the notation (e.g., ternary relationships, weak entities).

4.4 Conclusion Validity

A definition of conclusion validity could be the degree to which conclusions we reach about relationships in our data are reasonable. With regard to our experiment, proper tests were performed to statistically reject null hypotheses. In cases where differences were present but not significant, this was explicitly mentioned. Non-parametric tests were used in place of parametric tests where there were not the necessary conditions to use parametric tests. In particular, the Mann-Whitney test for unpaired analyses and the Wilcoxon test for paired analysis.

Concerning the survey questionnaires, they were mainly intended to get qualitative insights and were designed using standard ways and scales (Oppenheim 1992). This allowed us to use statistical analysis to analyse differences in the feedback provided by subjects with different levels of ability and experience.

5 Lessons Learned

The results achieved in the reported experiments provided us with a number of lessons learned:

- *UML class diagrams represent a notation easier to comprehend than ER diagrams independently of the subjects' experience.* The analysis also revealed that both subjects' ability and experience influence the comprehension level. Indeed, high ability students performed better than subjects with low ability, as well as master students achieved better results than bachelor students that also obtained better results than fresher students. It is important to note that the difference between the performances achieved by subjects with different levels of experience is much more evident when the task is performed using UML class diagrams. Our interpretation of such a result is that UML class diagrams are more concise than ER diagrams, as ER diagrams have more graphical elements than UML class diagrams (in particular, due to the graphical representation of attributes). Clearly, a concise notation well supports a top-down comprehension and, as also discussed in Ricca et al. (2007), subjects with high ability and high experience prefer to have a bird's eye of the problem and look at the details only if necessary. For this reason, a concise notation, like UML class diagrams, efficiently supports their approach, while the high number of graphical items in ER diagrams caused only noise. This consideration is further validated by the fact that subjects with high ability that performed the comprehension activities with UML class diagrams achieved better results than subjects with the same level of ability that used ER diagrams. Differently, no considerable difference was found between UML class diagrams and ER diagrams when subjects have low ability. All these considerations suggest that UML class diagrams are generally easier to

understand than ER diagrams. Moreover, a concise notation, like UML class diagrams, is not only recommended when software engineers are characterised by high ability and high experience, but also during the requirement elicitation phase that involves software engineers as well as customers, that probably do not have any background on design notation.

- *UML class diagrams do not provide better support than ER diagrams in the comprehension during maintenance.* This result suggests that even if ER is much more used than UML, the support given by the latter is at least equal to the support given by the former. Moreover, as expected subjects with high ability performed better than subjects with low ability, while no considerable difference was observed between graduate and undergraduate students. However, students with high ability obtained significant better results than students with low ability only when using UML class diagrams. Indeed, no considerable difference was found when the task were performed by subjects with different level of ability using ER diagrams. In particular, it seems that ER diagrams are able to bridge the gap between high and low ability subjects. The results achieved have also revealed that the experience do not influence the results achieved by subjects. Our interpretation of this (strange) result is that even if graduate students have more experience than undergraduate students on the design notations, they generally experienced forward engineering data modelling activities rather than comprehension, maintenance, and verification activities. In particular, they did not perform maintenance activities on data models in the context of university courses. Thus, they are unused to evolve an existing data model as also confirmed by the difficulties experienced by the subjects in performing the maintenance activities.
- *UML class diagrams better support bachelor students during the defect identification process with respect to ER diagrams.* This result confirms the findings achieved assessing the comprehension of the two notations. However, an unexpected result has been obtained when the experiment is replicated with master students. In this case, the two notations give the same support. Indeed, only 40% of the subjects performed better using UML class diagrams, and master students achieved better results than bachelor students only when using ER diagrams. A possible interpretation of these results is that when the experimentation was performed master students were attending an advanced course of Databases where ER was the used design notation. Thus, students probably had a better knowledge of ER with respect to UML class diagrams. However, this interpretation is not definitely supported by the results achieved when assessing the comprehension of the two notations, where master students achieved better results than bachelor students only when using UML class diagrams. Thus, this aspect has to be further investigated by replicating the experiment with different subjects and in different contexts.

6 Related Work

In the last two decades several papers have analysed, through controlled experiments or empirical studies or surveys, graphical notations supporting the software development process.

6.1 Empirical Comparison of ER and OO Models

To the best of our knowledge only four papers analyse and compare Entity-Relationship (ER), or its extensions, and Objected-Oriented (OO) models (Shoval and Shiran 1997; Shoval and Frummermann 1994; Bock and Ryan 1993; Palvia et al. 1992). Moreover, they analyse the results achieved by subjects performing comprehension activities and do not consider maintenance and verification activities. Table 17 compares the different results achieved investigating ER and UML class diagrams.

Shoval and Shiran (1997) compare Extended ER (EER) and OO data models from the point of view of design quality, where quality is measured in terms of correctness of the produced models, time to completely perform the design task, and designers' opinions. Differently, the goal of our empirical investigation was to compare ER and UML diagrams from a maintainer perspective to find out whether the use of UML diagrams provides better support during comprehension activities on data models. The results of the experimental comparison performed by Shoval and Shiran reveal that there are no significant differences between the two notations, except for the use of ternary and unary relationships for which EER models result to be better. Furthermore, the designers preferred to work with the EER models. Our results are quite different, showing that subjects involved in the controlled experiments performed comprehension activities with UML class diagrams significantly better than using the ER diagrams.

Palvia et al. (1992) carry out a comparison of OO and ER models from an end-user perspective with the aim of establishing which is more comprehensible. As in our empirical investigation, they measure comprehension on overall terms, not considering specific constructs, and the results of their investigation suggested that OO schemas are superior in this respect.

Shoval and Frummermann (1994) also perform a comparison of EER and OO diagrams taking into account the user comprehension. Differently from Palvia et al. (1992) and our empirical analysis, they separately examine the comprehension of various constructs of the analysed models. Their analysis reveals that EER schemas are more comprehensible for ternary relationships while for the other constructs no significant difference is found.

Bock and Ryan (1993) report a comparison of EER and OO models from a designer perspective, by examining the correctness of the design for several constructs of the considered diagram types. In particular, they find significant difference only in four cases (i.e., representation of attribute identifiers, unary 1:1 and binary m:n relationships) and no difference is found concerning the time to complete the tasks.

6.2 Empirical Analysis of the Influence of Subjects' Ability and Experience

Differently from our controlled experiments, the empirical analyses discussed in the previous section do not consider different levels of subjects' ability and experience. However, other experimentations carried out to compare other and different design notations take into account these factors.

Ricca et al. (2007) present the results of three experiments carried out to assess the effectiveness of UML stereotypes proposed by Conallen (1999) for the Web design in supporting comprehension activities. The performed replications of the

Table 17 Comparison of the results achieved comparing ER and UML class diagrams

	Goal	Perspective	Main result	Note
Shoval and Shiran (1997)	Design quality	Designer	No difference	EER better than UML considering ternary and unary relationships
Palvia et al. (1992)	Comprehension	Customers	UML better than ER	EER schemas are more comprehensible for ternary relationships
Shoval and Frummermann (1994)	Comprehension	Customers	No difference	ER better than UML considering the representation of attribute identifiers, unary 1:1 and binary m:n relationships
Bock and Ryan (1993)	Design	Designer	No difference	
Our work	Comprehension	Analyst and customers	UML better than ER	

first study allow authors to verify whether the experience and ability of subjects influence the comprehension level of the UML diagrams using stereotypes. Similar to our study, their experiments involved graduate and undergraduate subjects, with different levels of ability. The empirical results reveal that it is not possible to conclude that the use of stereotypes significantly improves the performance of subjects. However, differently from our experiments, the analysis has highlighted that experience and ability significantly interact with the treatment (i.e., using and not using UML stereotypes). In particular, subjects with low ability and experience achieved significant benefits from the use of stereotypes, while subjects with high ability and experience obtained comparable comprehension performances with or without using stereotypes. Thus, the authors conclude that the use of stereotypes reduces the gap between novices and experts. In our controlled experiments, both undergraduate and graduate students obtained significant better results in performing comprehension activities using UML class diagrams than using ER diagrams. Moreover, high ability subjects obtained significantly better performances than low ability subjects with all notations used. The use of stereotypes is also analysed by Kuzniarz et al. (2004), who obtain results similar to Ricca et al. (2007) regarding undergraduate students. However, the authors of this work do not consider and discuss the effect of different levels of experience and ability.

The controlled experiment performed by Cruz-Lemus et al. (2005) highlights that the use of composite states in UML statecharts improves the understandability of the diagrams when subjects have previous experience in using them. Similar results are obtained by Briand et al. (2005) who establish that training is required to achieve better results when UML is completed with the use of OCL (Object Constraint Language) (OMG 2005). They also highlight, as in Ricca et al. (2007), the interaction between ability and treatment in the case of defect detection. Reynoso et al. (2006) show that comprehensibility of OCL also depends on coupling.

The empirical study carried out by Hungerford and Eierman (2004) to compare the communication effectiveness of UML and traditional structured techniques takes into account three types of subjects: individuals with no knowledge of both modelling languages, individuals trained in one of the languages, and individuals more extensively trained on one of the languages. The results of the empirical investigation suggest that no difference was found in the ability of communicating information on system design for the first two sets of subjects. Differently, the analysis reveals that the use of UML diagrams during system modelling allows to improve the effectiveness of the communication for more extensively trained subjects.

6.3 Empirical Comparison of Other Design Models

Several studies discuss the results of empirical investigations comparing different design models (but not ER versus UML diagram). Also in this case the analysis is only based on the results achieved by subjects performing comprehension activities.

Wang (1996) compares the effectiveness of the Data Flow Diagram (DFD) (Gane and Sarson 1979) with the Object-Oriented Analysis (OOA) methods. The DFD results to be easier to learn for inexperienced participants, but with further training, the OOA method leads to more accurate answers. Thus, in some sense the author also considers different levels of experience. In our empirical analysis, we observe

a similar situation since *Graduate* subjects performed significantly better when they carried out comprehension activities with UML class diagrams.

Argwal et al. (1999) report the results of an empirical study comparing user comprehension of Object-Oriented (OO) and Process-Oriented (PO) models. The main difference between the two models is that OO diagrams focus on objects (i.e., structural aspects) while PO diagrams focus on processes (i.e., behavioural aspects). In their investigation, the authors take into account comprehension activities involving only structural aspects, only behavioural aspects, or a combination of structural and behavioural aspects. Two experiments have been conducted, each with a different application and a different group of subjects. The analysis of the results suggests that, for most of the simple questions, no significant difference is found in terms of comprehension. Differently, for most of the complex questions, the PO model is found to be easier to understand than the OO model.

Comprehension level is also taken into account by Gemino and Wand (1997), who compare three different notations: text, OO diagram, and DFD. The results of this study suggest that OO diagrams and DFD are better than text descriptions. The same authors also carry out an empirical study comparing two versions of ER model: one model where a grammars use optional properties and the other exploits mandatory properties and subtypes (Gemino and Wand 2005). The empirical results reveal that the use of mandatory properties can improve understanding even if the model seems more complex.

Purchase et al. (2004) compare the comprehension level of the syntax of the two types of ER notations, namely the Chen model (Chen 1976) and the SSADM notation (Downs et al. 1992). Overall, the results of their empirical analysis reveal that SSADM notation is better understood than Chen model since it is more concise, having fewer shapes and text on the page. Observe that, differently from our experiments, they consider students having no experience with ER notations as subjects. In a previous work, Purchase et al. (2001) also perform an empirical study for determining which variant of each of five UML class diagram constructs considered is the more suitable in terms of human performance. The subjects involved in the experiment are academic students and five experts. However, statistical analysis of experts is not reported due to the small number of observations. So, experts are asked to express only their opinion on the two notations and do not perform the comprehension activities. The study reveals that the variants of the notation indicated as less intuitive by the experts resulted to help better the academic students in the identification of errors in the diagrams.

The assessment of the usefulness of UML diagrams in comprehension activities is also the aim of other works. In particular, Torchiano (2004) verifies whether the use of static objects diagrams can improve the comprehension of software systems. The results of this preliminary empirical investigation suggest that object diagrams can significantly improve the comprehension for certain types of systems with respect to the use of only class diagrams. The author highlights that further analysis is needed to establish for what type of systems the use of object diagrams improves the comprehension. Otero and Dolado (2002) perform an empirical study aiming at comparing the semantic comprehension of three dynamic models of UML, namely sequence, collaborations, and state diagrams. The empirical results reveal that, overall, comprehension of UML dynamic models depends on the diagram type and on the complexity of the documents. However, they also find that software design

documents result to be more comprehensible when sequence diagrams are used to model dynamic behaviour.

7 Conclusion

The paper reported on the results of three sets of controlled experiments aimed at analysing whether or not UML class diagrams provide a better support during different kinds of activities on data models with respect to ER diagrams. Indeed, our conjecture is that UML class diagrams are easier to understand and maintain than ER diagrams due to their more concise graphical notation. The achieved results demonstrated that using UML class diagrams subjects achieved better comprehension levels. With regard to the support given by the two notations during maintenance activities the results demonstrated that the two notations give the same support, while in general UML class diagrams provide a better support with respect to ER diagrams during verification activities.

As it always happens with empirical studies, replications in different contexts, with different subjects and objects, is the only way to corroborate our findings. It would be interesting to consider alternative experimental settings in several respects, but maybe the most important one is the profile of the involved subjects. Replicating this study with students/professionals having a different background would be extremely important to understand how UML class diagrams influence the results of these different sub-populations.

Acknowledgements We would like to thank the anonymous reviewers for their detailed, constructive, and thoughtful comments that helped us to improve the presentation of the results in this paper. Special thanks are also due to the students who were involved in the experiment as subjects.

References

- Antoniol G, Canfora G, Casazza G, De Lucia A, Merlo E (2002) Recovering traceability links between code and documentation. *IEEE Trans Softw Eng* 28(10):970–983
- Argwal R, De P, Sinha AP (1999) Comprehending object and processes models: an empirical study. *IEEE Trans Softw Eng* 25(4):541–556
- Arisholm E, Sjöberg D (2004) Evaluating the effect of a delegated versus centralized control style on the maintainability of object-oriented software. *IEEE Trans Softw Eng* 30(8):521–534
- Baeza-Yates R, Ribeiro-Neto B (1999) Modern information retrieval. Addison-Wesley, Reading
- Basili VR, Selby RW, Hutchens DH (1986) Experimentation in software engineering. *IEEE Trans Softw Eng* 12(7):758–773
- Bock D, Ryan T (1993) Accuracy in modeling with extended entity relationship and object oriented data models. *J Database Manage* 4(4):30–39
- Briand L, Labiche Y, Di Penta M, Yan-Bondoc H (2005) An experimental investigation of formality in UML-based development. *IEEE Trans Softw Eng* 31(10):833–849
- Chen PP (1976) The entity-relationship model: toward a unified view of data. *ACM Trans Database Syst* 1(1):1–36
- Conallen J (1999) Building web applications with UML. Addison-Wesley object technology series. Addison-Wesley, Reading
- Conover WJ (1998) Practical nonparametric statistics, 3rd edn. Wiley, New York
- Cruz-Lemus JA, Genero M, Manso ME, Piattini M (2005) Evaluating the effect of composite states on the understandability of UML statechart diagrams. In: Proceedings of 8th ACM/IEEE international conference on model driven engineering languages and systems. Springer, Montego Bay, pp 113–125

- De Lucia A, Fasano F, Francese R, Tortora G (2004) ADAMS: an artefact-based process support system. In: Proceedings of 16th international conference on software engineering and knowledge engineering. KSI, Banff, pp 31–36
- De Lucia A, Gravino C, Oliveto R, Tortora G (2008a) An experimental comparison of ER and UML class diagrams for data modelling: experimental material. Technical report. www.sesa.dmi.unisa.it/reportUMLvsER.pdf
- De Lucia A, Gravino C, Oliveto R, Tortora G (2008b) Assessing the support of ER and UML class diagrams during maintenance activities on data models. In: Proceedings of the 12th European conference on software maintenance and reengineering. IEEE, Athens, Greece, pp 173–182
- De Lucia A, Gravino C, Oliveto R, Tortora G (2008c) Data model comprehension: an empirical comparison of ER and UML class diagrams. In: Proceedings of the 16th IEEE international conference on program comprehension. IEEE, Amsterdam, pp 93–102
- Devore JL, Farnum N (1999) Applied statistics for engineers and scientists. Duxbury
- Downs E, Clare P, Coe I (1992) Structured systems analysis and design method: application and context. Prentice Hall, Englewood Cliffs
- Gane C, Sarson T (1979) Structured systems analysis: tools and techniques. Prentice-Hall, Englewood Cliffs
- Gemino W, Wand Y (1997) Empirical comparison of objected oriented and dataflow models. In: Proceedings of international conference on information systems. ACM, Atlanta, pp 446–447
- Gemino A, Wand Y (2005) Complexity and clarity in conceptual modeling: comparison of mandatory and optional properties. *Data Knowl Eng* 55(3):301–326
- Henderson PB (2003) Mathematical reasoning in software engineering education. *Commun ACM* 46(9):45–50
- Hungerford BC, Eierman MA (2004) The communication effectiveness of system models using the UML versus structured techniques: a field experiment. *American Journal of Business* 20(2):35–43
- Juristo N, Moreno A (2001) Basics of software engineering experimentation. Kluwer Academic, Dordrecht
- Kitchenham B, Fry J, Linkman S (2003) The case against cross-over designs in software engineering. In: Proceedings of 11th annual international workshop on software technology and engineering practice, pp 65–67
- Kuzniarz L, Staron M, Wholin C (2004) An empirical study on using stereotypes to improve understanding on UML models. In: Proceedings of 12th IEEE international workshop on program comprehension. IEEE CS, Bari, pp 14–23
- Miller GA (1956) The magical number seven, plus or minus two: some limits on our capacity for processing information. *Psychol Rev* 63(2):81–97
- Navathe SB (1992) Evolution of data modeling for databases. *Commun ACM* 35(9):112–123
- OMG (2005) Object constraint language (OCL) specification, version 2.0. <http://www.omg.org/technology/documents/formal/uml.htm>
- Oppenheim AN (1992) Questionnaire design, interviewing and attitude measurement. Pinter, London
- Otero C, Dolado JJ (2002) An initial experimental assessment of the dynamic modelling in UML. *Empirical Software Engineering* 7(1):27–47
- Palvia P, Lio C, To P (1992) The impact of conceptual data models on end-user performance. *J Database Manage* 3(4):4–15
- Purchase HC, Wellanda R, McGill M, Colpoys L (2004) Comprehension of diagram syntax: an empirical study of entity relationship notations. *Int J Human-comput Stud* 61(2):187–203
- Purchase HC, Colpoys L, McGill M, Carrington D, Britton C (2001) UML class diagram syntax: an empirical study of comprehension. In: Proceedings of Australian symposium on information visualisation. Australian Computer Society, Sydney, pp 113–120
- Reynoso L, Genero M, Piattini M, Manso ME (2006) Does object coupling really affect the understanding and modifying of UML expressions? In: Proceedings of 21st annual ACM symposium on applied computing. ACM, Dijon, pp 1721–1727
- Ricca F, Di Penta M, Torchiano M, Tonella P, Ceccato M (2007) The role of experience and ability in comprehension tasks supported by UML stereotypes. In: Proceedings of 29th international conference on software engineering. IEEE Computer Society, Minneapolis, pp 375–384
- Rumbaugh J, Jacobson I, Booch G (2004) Unified modeling language reference manual. Addison-Wesley, Reading
- Shoval P, Frumermann I (1994) OO and EER conceptual schemas: a comparison of user comprehension. *J Database Manage* 5(4):28–38

- Shoval P, Shiran S (1997) Entity-relationship and object-oriented data modeling - an experimental comparison of design quality. *Data Knowl Eng* 21(3):297–315
- Torchiano M (2004) Empirical assessment of UML static object diagrams. In: *Proceedings of 12th international workshop in program comprehension*. IEEE Computer Society, Bari, pp 226–229
- Wang S (1996) Two MIS analysis methods: an experimental comparison. *J Educ Bus* 61(3):136–141
- Wohlin C, Runeson P, Host M, Ohlsson MC, Regnell B, Wesslen A (2000) *Experimentation in software engineering—an introduction*. Kluwer, Deventer
- Zimmermann T, Weissgerber P, Diehl S, Zeller A (2005) Mining version histories to guide software changes. *IEEE Trans Softw Eng* 31(6):429–445



Andrea De Lucia received the Laurea degree in Computer Science from the University of Salerno, Italy, in 1991, the MSc degree in Computer Science from the University of Durham, U.K., in 1996, and the PhD in Electronic Engineering and Computer Science from the University of Naples “Federico II”, Italy, in 1996. He is a full professor of Software Engineering and the Director of the International Summer School on Software Engineering at the Department of Mathematics and Informatics of the University of Salerno, Italy. Previously he was at the Research Centre on Software Technology (RCOST) of the University of Sannio, Italy. Prof. De Lucia is actively consulting in industry and has been involved in several research and technology transfer projects conducted in cooperation with industrial partners. His research interests include software maintenance, program comprehension, reverse engineering, reengineering, migration, global software engineering, software configuration management, workflow management, document management, empirical software engineering, visual languages, web engineering, and e-learning. He has published more than 100 papers on these topics in international journals, books, and conference proceedings. He has also edited books and special issues of international journals and serves on the editorial and reviewer boards of international journals and on the organizing and program committees of several international conferences in the field of software engineering. Prof. De Lucia is a member of the IEEE, the IEEE Computer Society, and the executive committee of the IEEE Technical Council on Software Engineering.



Carmine Gravino received the Laurea degree in Computer Science (cum laude) in 1999, and his PhD in Computer Science from the University of Salerno (Italy) in 2003. Since March 2006 he is assistant professor in the Department of Mathematics and Informatics at the University of Salerno. His research interests include software metrics and techniques to estimate web application development effort, software-development environments, design pattern recovery from object-oriented code, evaluation and comparison of notations, methods, and tools supporting software development and maintenance. He has published more than 40 papers on these topics in international journals, books, and conference proceedings.



Rocco Oliveto received (cum laude) the Laurea in Computer Science from the University of Salerno (Italy) in 2004. From October 2006 to February 2007 he has been a visiting student at the University College London, UK, under the supervisor of prof. Anthony Finkelstein. He received the PhD in Computer Science from the University of Salerno (Italy) in 2008. From July 2009 to September 2009 he has been a visiting researcher at the Polytechnique of Montreal, Canada. He is currently a research fellow at the Department of Mathematics and Informatics of the University of Salerno. Moreover, since 2005 he is also adjunct professor at the Faculty of Science of the University of Molise, Italy. His research interests include traceability management, information retrieval, software maintenance and evolution, program comprehension, empirical software engineering and software development effort estimation. He has published about 30 papers on these topics in international journals, books, and conference proceedings. He also serves on the reviewer boards of international journals and on the program committees of several international conferences in the field of software engineering. Dr. Oliveto is a member of IEEE and ACM.



Genoveffa Tortora received the Laurea degree in Computer Science from the University of Salerno, Italy, in 1978. Since 1990, she has been a full professor at University of Salerno, Italy, where she teaches database systems and fundamentals of computer science. In 1998, she was a founding member of the Department of Mathematics and Computer Science, acting as chair until October 2000. From 2000 to 2008, she was Dean of the Faculty of Mathematical, Natural and Physical Sciences, at the University of Salerno. She is author and coauthor of several papers published in scientific journals, books, and proceedings of refereed conferences, and is coeditor of two books. She is an associate editor and reviewer for international scientific journals. She has been program chair and program committee member in a number of international conferences. Her research interests include software engineering, visual languages, geographical information systems, and pictorial information systems. She is a senior member of the IEEE Computer Society.